

Making Language Models Version Aware

Anonymous ACL submission

Abstract

Large language models often rely on large-scale web corpora for pretraining; however, noise in these data can sometimes lead to unexpected consequences. For example, we found that a majority of OLMo-2 models’ errors on coding tasks were syntax errors—specifically mixing Python2 and Python3 syntax—rather than runtime errors or test case failures. This issue can be traced back to its pretraining corpus, StarCoder, which contains about 10% Python2 code snippet mixed with Python3. To mitigate this issue, we introduce a lightweight post-training pipeline that makes OLMo models version-aware for Python. Specifically, we construct a version-aware code instruction fine-tuning dataset using the OLMo’s original pre-training corpus. We extract code snippets from StarCoder and use AST parsing to determine whether each snippet belongs to Python2 or Python3. For each snippet, we then generate synthetic instructions that explicitly encode the version information, such as “generate a Python3 program to ...”. Fine-tuning with LoRA on only ~5K examples proves highly effective: it significantly boosts performance on coding tasks—for instance, HumanEval rises from 2.4% to 18.9% and MBPP from 0.0% to 16.8%. Surprisingly, it also maintains or even improves performance in other domains such as math, QA, and reading comprehension.

1 Introduction

Large language models (LLMs) trained on open-source corpora have shown impressive progress in code generation and reasoning. However, model behavior can vary widely depending on the quality and composition of their pretraining data. We take OLMo (Allen Institute for AI, 2024) as a case study because it is fully open-weight and transparent, allowing us to trace model failures directly to their data sources. Despite being pretrained on a StarCoder-derived corpus (Li et al., 2023),

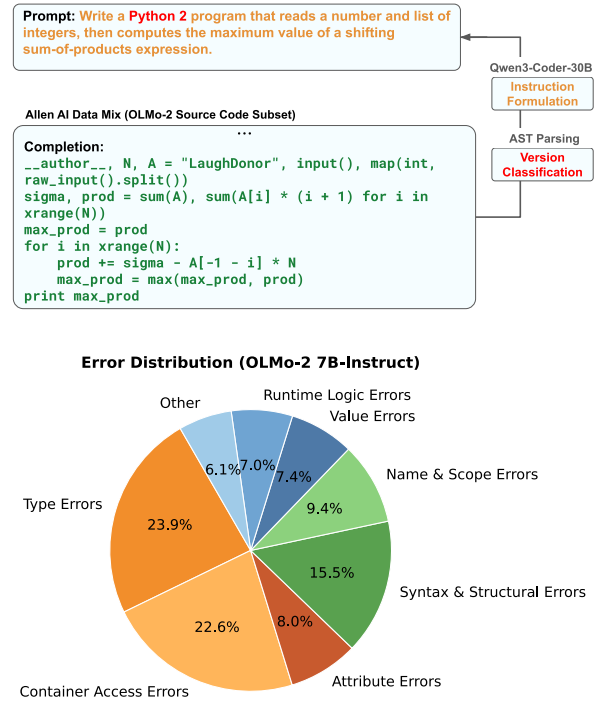


Figure 1: **Figure 1.** (a, top) Version-aware preprocessing pipeline. Raw Python code samples from the *Allen AI Data Mix* (derived from StarCoder / The Stack) are parsed using the Python 3 `ast` module and `typed_ast.ast27` for syntax-based version detection. Each snippet is labeled as python2 or python3, then reformulated into an instruction-completion pair using Qwen3-Coder-30B for finetuning. In this example, `raw_input()` exists only in Python 2; when a Python 3 interpreter encounters it, it raises a NameError. (b, bottom) Distribution of runtime error classes for OLMo-2 7B-Instruct on the LeetCode dataset. We group these error classes (1) Compiles but Fails at Runtime (orange), (2) Fails to Compile (green), and (3) Runs but Produces Wrong Output (blue). Most failures (~62%) stem from Python 2→3 mismatches, primarily type and container-access errors.

OLMo-2-1B-Instruct achieves only 2.4% pass@1 on HumanEval (Zheng et al., 2023) and 0.0% on MBPP (Austin et al., 2021), highlighting substantial headroom for improvement.

As shown in Figure 1, most model failures arise from Python 2→3 mismatches. Our error analysis reveals that roughly 60% of runtime and compila-

tion failures arise from Python 2→3 mismatches, use of `raw_input()` or outdated print syntax rather than algorithmic mistakes. When we trace these failures back to the pretraining corpus, we find mixed Python versions and unparseable fragments that disrupt syntactic coherence.

To mitigate this issue, we investigate two simple but effective preprocessing interventions on code data (1) AST-based programming language version classification: Using abstract syntax tree (AST) parsing to identify the correct Python version (2 vs. 3) for each snippet, ensuring syntactic validity under both interpreters. (2) Instruction Formulation: Creating natural language instruction-code pairs where instructions describe the functionality and requirements of the code, enabling supervised fine-tuning for code generation tasks across different Python versions. An example of the instruction-generation pipeline is provided in Figure 1.

We observe a mean absolute improvement of roughly 17% on coding benchmarks (HumanEval and MBPP) after fine-tuning on the 10k dataset. This corresponds to an average 2.6× relative gain over the base models, with the strongest improvement seen in larger models (13B: + 26%). The consistency of these gains across scales suggests that resolving syntactic ambiguity benefits both small and large models, reinforcing that even lightweight data interventions can yield large downstream improvements in code reasoning ability.

2 Related Work

Open models such as OLMo-2 (Tay and et al., 2024), Code LLaMA (Roziere et al., 2023), StarCoder (Li et al., 2023), and CodeGen (Nijkamp et al., 2023) have advanced open-source code generation through large-scale pretraining. These efforts primarily emphasize scale and instruction tuning. Our work complements them by showing that targeted *data quality improvements*—specifically Python version disambiguation and instruction augmentation—can further boost performance even without increasing model size.

Synthetic instruction generation has proven effective for aligning smaller models with frontier-level behavior. Wang et al. (2023) introduced *Self-Instruct*, where models bootstrap their own supervision, while Alpaca (Taori et al., 2023), WizardLM (Xu et al., 2023), and WizardCoder (Luo et al., 2023) use frontier-model-generated or evolved instructions to improve downstream usability.

We adopt this paradigm using Qwen3-Coder-30B-A3B-Instruct-FP8 (Team, 2025) to generate task-specific prompts for raw Python code, improving the instruction-following quality of open models.

Benchmarks such as CodeSearchNet (Husain et al., 2019), MBPP (Austin et al., 2021), APPS (Hendrycks et al., 2021), MultiPL-E (Cassano et al., 2022), DS-1000 (Lai et al., 2023), and HumanEval-X (Zheng et al., 2023) measure code generation and reasoning across programming languages. However, none explicitly address cross-version ambiguity within a language. We show that tagging and balancing Python 2 and Python 3 data can improve both stability and accuracy across multiple benchmarks.

Recent work highlights data quality as a key bottleneck in code modeling. BigCode (Project, 2023) emphasizes license and filtering rigor, while GraphCodeBERT (Guo et al., 2021) explores structural representations. Error analyses (e.g., HumanEval-X (Zheng et al., 2023)) find that noisy or inconsistent data limits cross-language generalization. We treat Python version mismatch as a concrete data-quality problem and demonstrate measurable gains from resolving it.

While prior work has focused on model scale and synthetic instructions, our results show that **fine-grained dataset engineering**—version disambiguation and model-assisted cleaning—offers a scalable path to improving open-source code models.

3 Method: Version-Aware Fine-tuning

Our goal is to correct syntactic inconsistencies in OLMo’s pretraining corpus through a lightweight, version-aware fine-tuning pipeline. At a high level, the method (i) classifies Python snippets by version using abstract syntax trees (AST), (ii) constructs version-aware instruction-completion pairs for supervised fine-tuning (SFT), and (iii) fine-tunes OLMo models on this dataset and evaluates them across eight standard code and reasoning benchmarks.

3.1 Data Preparation

We start from the StarCoder component of the OLMo-Mix-1124 corpus, focusing on files tagged as Python. After filtering for sequence length within OLMo-2’s 4K context window, we retain roughly ten million Python samples. Empirically,

about 4% of valid examples correspond to Python 2 and 96% to Python 3 (details in Appendix A.3).

AST-based Version Detection. Each code snippet is parsed using Python 3.10’s ast module; if parsing fails, we re-parse with Python 2.7 via typed_ast. Samples successfully parsed by Python 3 are labeled python3, those valid only under Python 2 are labeled python2, and those invalid under both are discarded. This step eliminates mixed-version fragments and establishes a clean, version-disambiguated corpus.

Instruction Construction. For a balanced subset of roughly two million samples, we synthesize natural-language instructions using Qwen3-Coder-30B. Prompts are few-shot and explicitly request instructions that reproduce the given code, with required inclusion of version keywords (python2 / python3). The resulting pairs form an SFT-ready dataset of instruction–completion examples.

3.2 Model Finetuning

Each training sample consists of an instruction prompt and its corresponding code completion. We finetune OLMo-2-1B-Instruct using standard supervised finetuning with a next-token cross-entropy loss. The complete training setup can be found in Appendix A.4). All experiments use balanced data (50% Python 2, 50% Python 3).

At inference time, evaluation prompts are phrased to mirror the training setup—for example, “Please generate a Python 3 program to solve this question.” This alignment ensures consistency between training and evaluation setups. Benchmark results are shown in Table 1.

3.3 Experimental Design

Baselines. We compare our approach against two baselines. (1) **OLMo (no fine-tuning)**: the original OLMo-2-Instruct model evaluated directly without any additional training. (2) **Version blind instruction finetuning**: a control variant trained on the same 10K cleaned dataset but with all python2 and python3 tags removed from the synthetic instructions. This comparison isolates the contribution of explicit version labeling beyond general syntactic cleaning.

Scaling Study. To identify the saturation point of data scaling under a fair and controlled protocol, we train models on datasets ranging from 1K to 100K samples. To examine the effect of dataset

size on retention and generalization, we vary the amount of training data from 1K to 100K samples while keeping the total number of gradient updates constant. This isolates the impact of data scale on downstream performance and allows us to probe potential “forgetfulness” when training on larger, noisier subsets.

Evaluation Benchmarks. We evaluate on eight benchmarks using the lm-eval-harness: HumanEval and MBPP (pass@1, 0-shot, temperature=0.2, $n = 20$ samples), GSM8K (exact match, 8-shot), MMLU (accuracy, 5-shot), MATH (exact match, 4-shot), and DROP, TriviaQA, and TruthfulQA-MC2 (F1, exact match, and MC2 respectively, all 0-shot).

4 Results

Across a sample of runtime failures, we find that the majority of errors (~62%) arise from Python 2 → 3 mismatches rather than algorithmic mistakes. In particular, type errors (24%) and container access errors (23%) dominate the distribution. Specific failure cases and detailed traces can be found in A.1.

We evaluate fine-tuned OLMo-2 models (1B, 7B, 13B) on eight benchmarks: HumanEval, MBPP, GSM8K, MMLU, DROP, TriviaQA, TruthfulQA, and MATH and report performance for base models, train size of 10k and train size of 100k.

4.1 Overall Performance

Overall, instruction fine-tuning with version awareness leads to substantial improvements on code generation tasks. For example, HumanEval performance for the 13B model rises from 0.116 to 0.331 (+185% relative gain). Interestingly, gains are not strictly monotonic with dataset size: while 10k samples boost accuracy, 100k samples sometimes lead to marginal declines (e.g., HumanEval 13B: 0.331 → 0.311), suggesting diminishing returns or mild overfitting. On reasoning-oriented tasks such as GSM8K and MMLU, performance is preserved within 1–2 points, confirming that code-focused fine-tuning does not induce catastrophic forgetting. This indicates that version disambiguation can improve coding ability without sacrificing general reasoning.

We also observe evidence of *forgetfulness*: models trained longer using our larger 100k dataset occasionally underperform compared to the smaller 10k dataset. For instance, HumanEval accuracy for

Table 1: Benchmark results for **OLMo-2 Instruct** models without fine-tuning and after fine-tuning on Python instruction–response pairs of size **10k** or **100k**. “No Finetuning” corresponds to released checkpoints: allenai/OLMo-2-0425-1B-Instruct, allenai/OLMo-2-1124-7B-Instruct, allenai/OLMo-2-1124-13B-Instruct. Higher is better. **Bold** = best per model size across train sizes. HumanEval/MBPP report pass@1; GSM8K (8-shot) and MATH are exact match; MMLU is 5-shot accuracy; DROP is F1; TriviaQA is exact match; TruthfulQA is MC2.

Model	Train	Coding / Program Synthesis		Mathematical Reasoning		General Knowledge & Reading Comprehension			
		HumanEval	MBPP	GSM8K	MATH	MMLU	DROP	TriviaQA	TruthfulQA
OLMo-2 1B	No Finetuning	0.024	0.000	0.646	0.000	0.348	0.100	0.105	0.488
	Version Blind Finetuning	0.128	0.116	0.555	0.010	0.339	0.140	0.0550	0.419
	Version Aware Finetuning 10k	0.146	0.154	0.568	0.009	0.348	0.151	0.105	0.424
	Version Aware Finetuning 100k	0.116	0.136	0.526	0.006	0.321	0.133	0.043	0.418
OLMo-2 7B	No Finetuning	0.152	0.056	0.771	0.074	0.607	0.145	0.421	0.565
	Version Blind Finetuning	0.353	0.274	0.729	0.0716	0.607	0.443	0.443	0.513
	Version Aware Finetuning 10k	0.371	0.297	0.787	0.092	0.703	0.461	0.462	0.534
	Version Aware Finetuning 100k	0.332	0.292	0.772	0.092	0.610	0.473	0.476	0.534
OLMo-2 13B	No Finetuning	0.116	0.000	0.815	0.095	0.651	0.126	0.270	0.642
	Version Blind Finetuning	0.311	0.304	0.801	0.103	0.655	0.313	0.500	0.564
	Version Aware Finetuning 10k	0.333	0.324	0.816	0.099	0.660	0.337	0.524	0.661
	Version Aware Finetuning 100k	0.334	0.324	0.791	0.099	0.657	0.328	0.521	0.652

OLMo-2 13B peaks at 0.331 with 10k samples, but drops slightly to 0.311 with 100k. To mitigate this effect, we follow the strategy of capping training steps at the point of convergence observed in the 10k runs. Concretely, for the 7B model, the best checkpoint was reached at ~ 4500 steps on the 10k dataset; we reuse this stopping point when scaling to 100k. This early stopping heuristic stabilizes performance and prevents the degradation that can arise from over-training on larger datasets. Table 1 summarizes results across all benchmarks.

The two baselines provide an important context for these gains. The “OLMo (no fine-tuning)” baseline, representing the original instruct-tuned models, shows poor performance on coding benchmarks. For instance, the 1B and 13B models score 0.000 on MBPP, indicating a complete lack of coding ability. The “Version blind instruction finetuning” baseline demonstrates that simply cleaning the data and fine-tuning on instruction-completion pairs yields a substantial boost on its own. For the 7B model, this version-blind tuning improves MBPP from 0.056 to 0.274. However, the superior performance of the version-aware models across nearly all coding metrics confirms that explicit version disambiguation is a key contributor to the final performance, rather than the gains being solely an artifact of additional training on a cleaner dataset.

4.2 Per-Model Trends

Figures 6, 7, and 8 visualize performance improvements for each model size. We observe three main trends: (1) Coding benchmarks (HumanEval, MBPP, DROP) show the largest gains from curation. (2) General reasoning benchmarks (GSM8K, MMLU, TruthfulQA) are largely preserved, indicating no catastrophic forgetting. (3) As expected,

mid and large scale models (7B, 13B) benefit most

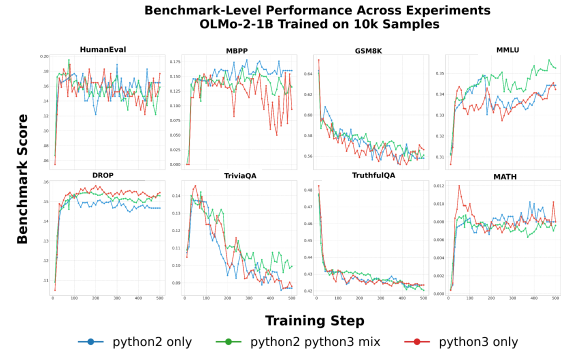


Figure 2: Performance of OLMo models on eight benchmarks after version aware instruction fine-tuning. We observe improvements across code generation tasks (HumanEval, MBPP) and general reasoning benchmarks. Version aware instruction fine-tuning yields the largest relative gains on coding tasks, improving HumanEval and MBPP by +16–17 percentage points on average.

5 Conclusion

A natural next step is to extend these insights beyond coding tasks. In particular, reasoning-heavy domains such as MATH remain challenging, and it is an open question whether similar data-centric interventions—careful curation, targeted augmentation, and language-version disambiguation—can also improve models’ reasoning reliability. Exploring this connection between data quality and reasoning ability will be central to future work.

6 Limitations

Our experiments evaluate the OLMo-2 instruction-tuned models (1B, 7B, 13B). This setup isolates the effect of version disambiguation within a single open model family. While OLMo baselines are relatively weak on coding tasks, the observed improvements are likely to generalize—and potentially amplify—in larger or stronger models.

Version disambiguation and model-assisted instruction generation improve the **alignment between training data and downstream tasks**, rather than exploiting model-specific features. Such alignment typically yields greater returns: larger models are more sensitive to fine-grained data structure and can better internalize systematic cues (e.g., Python 2 vs. Python 3 distinctions). We expect similar or even larger relative gains when applying these methods to high-capacity architectures such as LLaMA, Gemma, or Qwen-Coder.

6.1 Programming Language Scope

Although we demonstrate our approach on Python, the same AST-based classification and instruction augmentation framework can be readily applied to other programming languages with well-defined grammars (e.g., Java, C++, or JavaScript). However, for clarity and reproducibility, we focus on Python tasks in this work and have not empirically validated transfer beyond Python.

References

Allen Institute for AI. 2024. Olmo-2-0425-1b-instruct. <https://huggingface.co/allenai/OLMo-2-0425-1B-Instruct>. Open-source instruction-tuned model released by AllenAI on Hugging Face.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. [Program synthesis with large language models](#). In *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*.

Federico Cassano, Antonio Mastropaolo, Luca Ponzanelli, and et al. 2022. Multipl-e: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227*.

Daya Guo, Shuo Ren, Shuai Lu, and et al. 2021. Graphcodebert: Pre-training code representations with data flow. In *International Conference on Learning Representations (ICLR)*.

Dan Hendrycks, Steven Basart, Saurav Kadavath, and et al. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*.

Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. [Code-searchnet challenge: Evaluating the state of semantic code search](#). In *Proceedings of the NeurIPS Workshop on ML for Systems*.

Yujia Lai, Kevin Yang, Ziniu Xu, and et al. 2023. Ds-1000: A natural and reliable benchmark for data science code generation. *arXiv preprint arXiv:2211.11501*.

Raymond Li, Loubna Ben Allal, Denis Kocetkov, and et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.

Ziheng Luo, Canwen Xu, Qianhui Xie, and et al. 2023. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, and et al. 2023. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*.

BigCode Project. 2023. The bigcode project: Open scientific research on large language models for code. *arXiv preprint arXiv:2305.06161*.

Baptiste Roziere, Loubna Ben Allal, Raymond Li, and et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, and et al. 2023. [Stanford alpaca: An instruction-following llama model](#). *arXiv preprint arXiv:2303.16199*.

Yi Tay and et al. 2024. Olmo 2: Scaling open language models. *arXiv preprint arXiv:2407.16833*.

Qwen Team. 2025. Qwen3-coder-30b-a3b-instruct-fp8. <https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct-FP8>. Accessed: YYYY-MM-DD.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, and et al. 2023. [Self-instruct: Aligning language models with self-generated instructions](#). *arXiv preprint arXiv:2212.10560*.

Canwen Xu, Ziheng Luo, Ming Zhong, and et al. 2023. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*.

Xin Zheng, Yanan Li, Qi Zhu, and et al. 2023. Codegeex: A pre-trained model for code generation with multilingual evaluation on humaneval-x. *arXiv preprint arXiv:2303.17568*.

A Annotation Agreement

To validate the quality of our labeling pipeline, we computed inter-rater agreement (Cohen’s κ) across human annotators, GPT-5 annotations, and AST-based labeling heuristics. Table 2 shows results for a sample of 50 snippets.

Comparison	κ Score
Human 1 – Human 2	0.72
Human 1 – Model (GPT-5)	0.68
Human 2 – Model (GPT-5)	0.66
Human 1 – AST	0.70
Human 2 – AST	0.65

Table 2: Inter-rater agreement (Cohen’s κ) across humans, GPT-5, and AST heuristics.

For larger-scale validation, we compared AST vs. GPT-5 annotations on 10K samples, yielding $\kappa = 0.76$. This suggests that AST-based heuristics are sufficiently reliable to be used for large-scale disambiguation when human annotation is infeasible. We release GPT-5 annotations for 100K samples as part of our dataset contribution.

A.1 Failure Cases from OLMo-2 7B Instruct: Python 2 vs. Python 3 Ambiguity

A major source of error in the OLMo-2 7B instruct model is its inability to reliably disambiguate between Python 2 and Python 3 syntax.

To better illustrate these patterns, we present representative examples of failure cases drawn from LeetCode tasks, highlighting recurring artifacts such as outdated print syntax, legacy iteration methods, and Unicode string prefixes. These examples complement the quantitative distribution by showing how seemingly small syntactic mismatches can systematically undermine correctness. Since LeetCode test cases are executed in Python 3, outputs containing Python 2 constructs either fail at runtime or silently return incorrect results. Below we highlight common patterns and concrete generations from our evaluation set (temperature = 0.3, top- p = 0.95).

Common patterns. We frequently observe: (i) print statements without parentheses (syntax error in Python 3), (ii) use of xrange instead of range (undefined in Python 3), (iii) legacy dictionary iteration methods such as iteritems() (removed in Python 3), (iv) raw_input() instead of input() (raises NameError), and (v) old-style exception

syntax (except E, e:, invalid in Python 3). These issues lead to compilation/runtime errors or subtle semantic differences.

LeetCode #3305: Count of Substrings Containing Every Vowel and K Consonants I **Detected Python 2 artifact:** Unicode string prefix (u’...’) in the generated code. While valid in Python 2, this prefix is unnecessary in Python 3 and sometimes mishandled by downstream environments.

Model output (truncated):

```
class Solution:
    def countOfSubstrings(self, word: str, k:
        int) -> int:
        def valid_substring(s: str) -> bool:
            vowels = set('aeiou')
            all_vowels = all(c in s for c in
                vowels)
            return all_vowels and len(set(c for
                c in s if c not in vowels)) <= k

        def count_substrings(word: str, k: int)
            -> int:
            n = len(word)
            substrings = [word[i:j] for i in
                range(n) for j in range(i+1,
                    n+1)]
            return sum(1 for s in substrings if
                valid_substring(s))

        return count_substrings(word, k)
```

LeetCode feedback:

(no feedback text; wrong answer)

Although the algorithm is structurally correct, the legacy Unicode literal usage prevents successful execution under strict Python 3 judging.

LeetCode #3306: Count of Substrings Containing Every Vowel and K Consonants II **Detected Python 2 artifact:** again the use of Unicode string prefixes. This shows that the model consistently defaults to Python 2 conventions when generating string-handling code.

Model output (truncated):

```
class Solution:
    def countOfSubstrings(self, word: str, k:
        int) -> int:
        vowels = set('aeiou')
        consonants = set('bcdfghjklmnpqrstvwxyz')
        def count_substrings(word, k):
            count = 0
            start = 0
            while start < len(word):
                valid = True
                i = start
                while i < len(word) and valid:
                    if word[i] in vowels:
                        valid = True
                    elif word[i] in consonants
                        and count < k:
                        count += 1
```

```

490         else:
491             valid = False
492             i += 1
493         if valid:
494             count += 1
495     ... (truncated) ...

```

LeetCode feedback:

(no feedback text; wrong answer)

Here again, the logical structure is plausible, but the reliance on outdated syntax conventions leads to failing test cases. These qualitative examples reinforce our quantitative findings: many apparent coding errors arise not from flawed reasoning but from outdated syntax.

Disambiguating Python versions during training directly mitigates these failure modes, aligning model outputs with the execution environment.

A.2 Results: In Depth Analysis

To take a closer look at how the model’s general performance is effected by fine tuning, we evaluated OLMo-2-0425-1B-Instruct model every 10 steps during fine tuning.

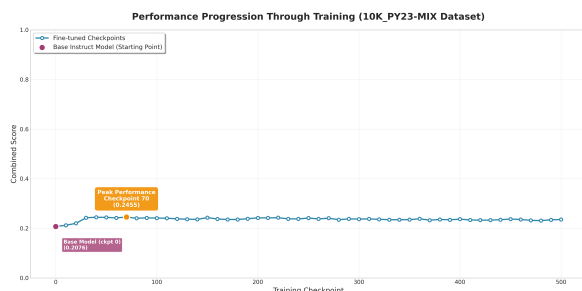


Figure 3: Average benchmark performance of OLMo-2-0425-1B-Instruct across checkpoints

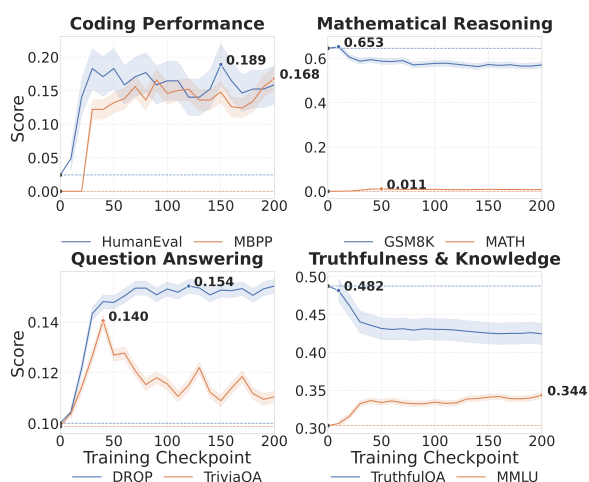


Figure 4: Individual benchmark performance of OLMo-2-0425-1B-Instruct across checkpoints

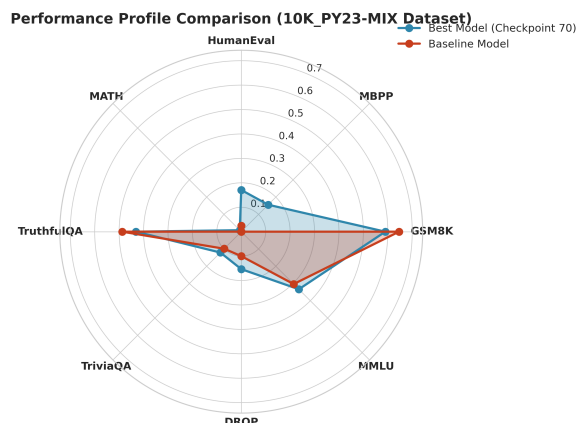


Figure 5: Radar graph showing the non fine tuned model against the best checkpoint

A.3 Data Preparation

We begin with the StarCoder component of the OLMo-Mix-1124 pretraining corpus, which contains 61.6 million documents. We identify Python files by filtering the corpus for samples tagged with “python” in the dataset’s metadata. This filtering yields 11,265,238 unique samples. To ensure compatibility with OLMo-2’s 4096-token context window, we filter samples exceeding 3796 tokens (reserving 300 tokens for special tokens and instructions), retaining approximately 10,649,867 unique samples.

Preprocessing Pipeline. We validate each sample using Python’s AST parser. First, we attempt parsing with Python 3.10’s ast module. Samples that fail are then tested with Python 2.7’s parser via the typed_ast package. Samples that pass Python 3 validation are labeled py3. Those passing only Python 2 validation are labeled py2. Samples failing both are discarded. This yields 10,025,402 valid Python 3 samples (94.14%), 425,314 valid Python 2 samples (3.99%), and 199,151 syntactically invalid samples (1.87%) that are removed.

Instruction Dataset Construction. We generate synthetic instructions for 1,976,448 samples using Qwen3-Coder-30B-A3B-Instruct-FP8. Our prompting strategy includes few-shot examples and explicit rules requiring the model to produce natural language requests that would result in the given code, with mandatory inclusion of version-specific keywords (python2 or python3). This produces 186,489 Python 2 instruction-response pairs and 1,728,595 Python 3 pairs.

A.4 Model Fine-Tuning

We fine-tune OLMo-2-1B-Instruct using LoRA with rank $r = 64$ and $\alpha = 128$. Training uses a learning rate of 5×10^{-5} , global batch size of 64, weight decay of 0.01, gradient clipping at 1.0, and dropout of 0.05. We train on 4 A100 80GB GPUs using a data-parallel strategy with mixed-precision (bfloat16) and gradient checkpointing. With a global batch size of 64, the per-device batch size was 16. For our experiments, we create balanced datasets (50% Python 2, 50% Python 3) at multiple sizes using seed 42 and the sampling script in the supplementary materials.

Scaling Study. To identify the saturation point of data scaling under a fair and controlled protocol, we train models on datasets ranging from 1K to 100K samples. Each model is trained for a fixed budget of approximately 1563 gradient steps, which is equivalent to one full epoch over the 100K dataset. This ensures that each experimental condition is exposed to the same number of model updates, making performance differences directly attributable to dataset size rather than training duration.

Saturation Analysis. We retrain on the 10K dataset while saving checkpoints every 10 training steps for the first 500 steps. We evaluate each checkpoint across all benchmarks to identify the precise point at which performance plateaus.

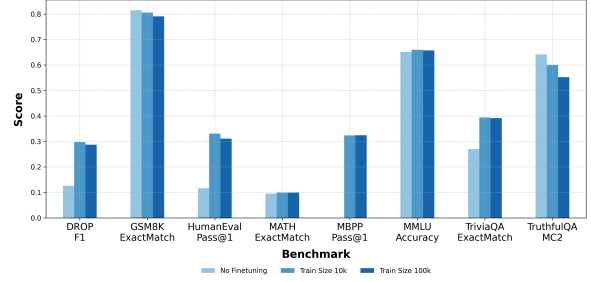


Figure 8: Performance of OLMo-2 13B across benchmarks.

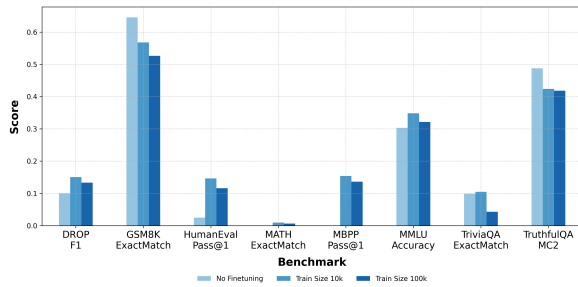


Figure 6: Performance of OLMo-2 1B across benchmarks.

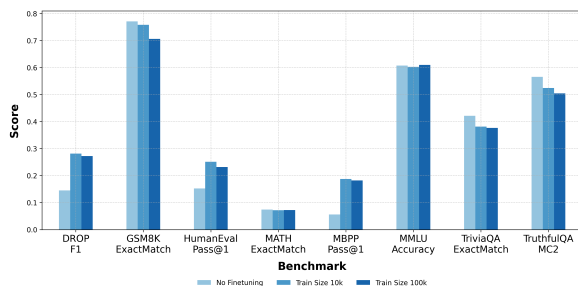


Figure 7: Performance of OLMo-2 7B across benchmarks.